

04-计算思维、流程图与 IPO 模型

编程不是一上来就敲代码。真正重要的是先把问题想清楚、拆清楚、写成步骤。本节学习计算思维、流程图和 IPO 模型，为后面的图形化编程和 Python 打基础。

一、学习目标

学完本节，你应该能做到：

1. 理解分解、抽象、模式识别、算法四种计算思维方法。
 2. 能把一个大任务拆成几个小任务。
 3. 能找出问题中的关键数据和规则。
 4. 能用自然语言写出算法步骤。
 5. 能用 IPO 模型说明程序的输入、处理和输出。
 6. 能读懂并画出简单流程图。
 7. 能理解顺序、分支、循环三种基本结构。
-

二、什么是计算思维

计算思维是一种解决问题的方法。它不是只属于程序员，生活中也经常用到。

可以用一句话理解：

把复杂问题拆开，找出关键规律，设计清楚步骤，再不断测试和改进。

计算思维主要包括四个方法。

方法	含义	例子
分解	把复杂问题拆成小问题	做班级通讯录：收集信息、整理表格、排序、保存
抽象	保留关键内容，去掉无关细节	成绩系统只关心姓名、学号、成绩
模式识别	找出相同或重复规律	多个文件都按“日期-科目-主题”命名
算法	解决问题的明确步骤	判断成绩是否及格的步骤

三、分解：把大问题拆小

例子：制作校园文明宣传网页

如果直接说“做一个网页”，任务太大。

可以分解成：

1. 确定主题：校园文明
2. 收集资料：文字、图片、标语
3. 设计结构：首页、文明行为、不文明行为、倡议
4. 编写 HTML
5. 编写 CSS 美化页面
6. 加一个按钮交互
7. 测试图片和链接
8. 整理文件并提交

这样每一步都更容易完成。

分解练习方法

拿到任务后问：

1. 这个任务最后要得到什么？
 2. 为了得到它，要先准备什么？
 3. 哪些步骤必须按顺序做？
 4. 哪些步骤可以分开做？
 5. 哪一步最容易出错？
-

四、抽象：抓住关键

抽象就是去掉暂时不重要的细节，只保留解决问题需要的信息。

例子：判断学生是否及格

现实中一个学生有很多信息：

- 姓名
- 年龄
- 班级
- 身高
- 兴趣
- 成绩
- 座位

但判断是否及格时，最关键的是：

成绩

规则是：

成绩 ≥ 60 : 及格
成绩 < 60 : 不及格

所以可以抽象成:

输入: score
处理: 判断 score 是否大于等于 60
输出: 及格或不及格

五、模式识别：找规律

模式识别就是发现重复出现的结构或规律。

例子 1：文件命名规律

这些文件：

2026-信息科技-第1次作业.docx
2026-信息科技-第2次作业.docx
2026-信息科技-第3次作业.docx

规律是：

年份-科目-第几次作业.扩展名

例子 2：循环规律

如果要输出 1 到 10，不需要写十句完全重复的话。

规律是：

从 1 开始
每次输出当前数字
数字加 1
直到超过 10

这就是循环思想。

六、算法：明确步骤

算法是解决问题的明确步骤。

一个好算法应该：

1. 每一步清楚。
2. 顺序明确。
3. 可以执行。
4. 有结束条件。
5. 能得到结果。

示例：判断成绩是否及格

自然语言算法：

1. 输入成绩
2. 如果成绩大于或等于 60
3. 输出“及格”
4. 否则
5. 输出“不及格”
6. 结束

示例：自动售货机卖饮料

1. 用户选择饮料
2. 显示价格
3. 用户付款
4. 判断付款是否足够
5. 如果付款足够，出饮料并找零
6. 如果付款不足，提示继续付款或取消
7. 结束

七、IPO 模型

IPO 是三个英文单词的缩写：

Input → Process → Output
输入 → 处理 → 输出

这是理解程序最常用的方法。

例子 1：问候程序

问题	回答
----	----

输入	用户输入姓名
----	--------

处理	把“你好,”和姓名拼接起来
----	---------------

输出 显示问候语

例子 2：购物总价计算

问题 回答

输入 商品单价、购买数量

处理 单价 $\times$ 数量

输出 总价

例子 3：奇偶数判断

问题 回答

输入 一个整数

处理 判断除以 2 的余数是否为 0

输出 奇数或偶数

写程序前先填 IPO 表

以后写代码前，可以先填：

项目	内容
任务名称	
输入	
处理	
输出	
可能出错的地方	
测试数据	

八、顺序、分支、循环

几乎所有基础程序都离不开三种结构。

1. 顺序结构

顺序结构就是从上到下，一步一步执行。

生活例子：

- 刷牙
1. 拿牙刷
 2. 挤牙膏

3. 刷牙
4. 漱口

编程例子：

1. 输入姓名
2. 输入年龄
3. 输出自我介绍

2. 分支结构

分支结构就是根据条件选择不同操作。

生活例子：

如果下雨，就带伞
否则，不带伞

成绩判断例子：

如果成绩 ≥ 60
 输出“及格”
否则
 输出“不及格”

3. 循环结构

循环结构就是重复执行一组步骤。

生活例子：

每天背 10 个单词，连续背 7 天

输出 1 到 10 的步骤：

1. 数字从 1 开始
2. 输出当前数字
3. 数字加 1
4. 如果数字不超过 10，回到第 2 步
5. 否则结束

九、流程图入门

流程图用图形表示步骤。

初学阶段不需要画得很复杂，先掌握常用符号。

符号	含义	用法
开始/结束	流程开始或结束	写“开始”“结束”
处理框	执行一个操作	计算、赋值、显示
输入/输出框	输入或输出数据	输入成绩、输出结果
判断框	根据条件选择	成绩是否 ≥ 60
箭头	表示执行顺序	指向下一步

如果不会画标准图形，也可以先用文字缩进表示流程。

登录流程示例

```
开始
输入用户名和密码
判断用户名和密码是否正确
    如果正确：显示登录成功
    如果错误：显示登录失败
结束
```

成绩判断流程示例

```
开始
输入成绩 score
如果 score  $\geq 60$ 
    输出“及格”
否则
    输出“不及格”
结束
```

十、调试思维

调试就是找出错误并修改。

程序或流程出错很正常。关键是不要慌，要有步骤。

调试基本步骤

```
复现问题
→ 找到位置
→ 分析原因
```

- 修改一处
- 再次测试
- 记录结果

例子：成绩判断错误

现象：输入 60，程序输出“不及格”。

可能原因：

1. 条件写成了 `score > 60`，没有包含 60。
2. 输入的成绩被当成文本，没有转换成数字。
3. 判断顺序写错。

排查方法：

1. 用 59、60、61 三个数据测试。
 2. 重点检查 60 这个边界。
 3. 一次只修改一个地方。
 4. 修改后重新运行同样测试。
-

十一、本节练习

练习 1：分解一个生活任务

选择一个任务：

- 整理书包。
- 完成一份手抄报。
- 制作班级通讯录。
- 准备一次小组展示。

把它拆成 5 到 8 个步骤。

要求：

1. 步骤要清楚。
2. 有些步骤要体现先后顺序。
3. 写出最容易出错的一步。

练习 2：填写 IPO 表

任务：判断一个数是否为偶数。

项目	内容
----	----

输入
处理
输出
测试数据

提示：偶数除以 2 的余数是 0。

练习 3：设计自动售货机算法

要求写出自然语言步骤，至少包含：

1. 选择商品。
2. 显示价格。
3. 付款。
4. 判断金额是否足够。
5. 出货或提示继续付款。
6. 结束。

再回答：

- 输入是什么？
- 处理是什么？
- 输出是什么？
- 哪一步是分支结构？

练习 4：画成绩等级流程

成绩等级规则：

分数范围	等级
------	----

90 到 100	优秀
----------	----

80 到 89	良好
---------	----

60 到 79	合格
---------	----

0 到 59	需要努力
--------	------

要求：

1. 先写 IPO 表。
2. 再写自然语言算法。
3. 最后画流程图或用文字缩进表示。
4. 准备测试数据：100、90、89、80、79、60、59、0。

练习 5：循环描述

用自然语言描述“输出 1 到 10”。

必须包含：

1. 从哪个数开始。
2. 每次做什么。
3. 数字如何变化。
4. 什么时候继续。
5. 什么时候结束。

十二、常见问题与对策

问题	对策
不知道怎么开始	先写“输入、处理、输出”
步骤太乱	用编号，一步只写一件事
把很多细节都写进去	先抽象，只保留关键数据和规则
分支条件写不清	用“如果……否则……”句式
循环不知道怎么结束	明确起点、重复内容、变化量、结束条件
流程图画不标准	先用文字缩进，理解比形式更重要

十三、本节达标清单

- ☐ 我能解释分解、抽象、模式识别和算法。
- ☐ 我能把一个大任务拆成几个小步骤。
- ☐ 我能找出问题中的关键数据和规则。
- ☐ 我能用 IPO 模型描述程序。
- ☐ 我能区分顺序、分支、循环。
- ☐ 我能用自然语言写出算法步骤。
- ☐ 我能读懂简单流程图。
- ☐ 我知道出错后要复现、定位、修改、测试和记录。

如果你能做到这些，再学习 Scratch 或 Python 时，就不会只是在照抄代码，而是在用代码表达自己的思路。